



RAPPORT DE PROJET
PROGRAMMATION EN LANGAGE C

LA BATAILLE NAVALE
Conception et développement d'un jeu modulaire

Réalisé par :

Enzo OLIVIER
Alix PESTOURIE
Julien STEVENART

Sous la supervision de :
M. SART

Année Universitaire 2025 – 2026

Table des matières

1	Introduction	3
1.1	Contexte du projet	3
1.2	Présentation du sujet	3
1.3	Objectifs techniques et fonctionnalités	3
1.4	Organisation de l'équipe	4
2	Architecture et Planification	5
2.1	Organisation Modulaire (Arborescence)	5
2.2	Planification et Répartition des tâches	6
3	Structures de Données et Logique Cœur	8
3.1	La structure Joueur : La clé de voûte	8
3.2	Algorithme de Placement des Bateaux	8
4	Fonctionnalités Avancées	10
4.1	L'Intelligence de l'Ordinateur	10
4.2	La Persistance des Données (Sauvegarde)	11
5	Interface et Ergonomie	12
5.1	Gestion de l'affichage console	12
5.2	Sécurisation des Entrées	13
6	Bilan et Retour d'expérience	14
6.1	Bilan des réalisations	14
6.2	Apports pédagogiques et techniques	14
6.3	Retour sur le travail d'équipe	14
6.4	Perspectives d'amélioration	14

1 Introduction

1.1 Contexte du projet

Ce projet a été réalisé dans le cadre de l'unité d'enseignement d' **Algorithmique avancée et programmation** de notre cursus universitaire. L'objectif pédagogique principal était de concevoir, développer et documenter un jeu complet et autonome en langage C, en mettant en œuvre de manière concrète les notions avancées abordées en cours et en travaux pratiques.

Le cahier des charges nous imposait plusieurs contraintes techniques fortes : une gestion rigoureuse de la mémoire (allocation dynamique via `malloc/free`), l'utilisation de structures complexes, la manipulation de fichiers binaires pour la sauvegarde des données et le chargement de parties, ainsi qu'une compilation modulaire (fichiers `.h` et `.c` séparés). De plus, le code devait être compilé sans avertissements avec les drapeaux `-Wall -Wextra`, garantissant ainsi sa propreté et sa portabilité.

Nous avons également pour consigne de travailler en groupe de trois étudiants. Cette dimension collaborative était essentielle pour nous confronter aux réalités du développement : répartition des modules, gestion des conflits de code, harmonisation des styles d'écriture et communication technique.

1.2 Présentation du sujet

Nous avons choisi de développer une version numérique fidèle du célèbre jeu de stratégie : la **Bataille Navale**.

Le principe du jeu repose sur l'affrontement de deux amiraux. Chaque joueur dispose d'une grille de 10×10 cases (coordonnées de A1 à J10) et d'une flotte identique composée de 5 navires réglementaires :

- 1 Porte-avion (5 cases)
- 1 Croiseur (4 cases)
- 2 Contre-torpilleurs (3 cases)
- 1 Torpilleur (2 cases)

Le but est de couler l'intégralité de la flotte adverse en devinant les positions des navires ennemis. Le jeu intègre la notion de "Brouillard de Guerre" : chaque joueur possède deux grilles, l'une défensive où il voit ses propres navires, et l'une offensive où il note le résultat de ses tirs (Touché, Coulé, ou Dans l'eau).

Notre implémentation propose deux modes de jeu distincts : un mode "Joueur contre Joueur" (1vs1) nécessitant de se passer le terminal, et un mode "Joueur contre Ordinateur" où l'utilisateur affronte des algorithmes de difficulté croissante.

1.3 Objectifs techniques et fonctionnalités

Au-delà de la simple transposition des règles, nous nous sommes fixés des objectifs pour rendre le projet abouti :

- **Architecture Modulaire** : Pour éviter un code monolithique impossible à maintenir, nous avons divisé le projet en modules logiques indépendants (`jeu`, `grille`,

bateau, affichage, sauvegarde). Cela facilite le débogage et le travail en parallèle.

- **Gestion avancée de la mémoire** : Toutes les structures principales (Joueurs, Grilles, Flottes) sont allouées dynamiquement. Nous avons porté une attention particulière à la libération de la mémoire en fin de partie pour assurer un programme sans fuite mémoire.
- **Expérience Utilisateur (UX) en Console** : Bien que contraints à l’affichage texte, nous avons soigné l’ergonomie : utilisation de codes couleurs ANSI (Bleu pour l’eau, Rouge pour les dégâts), nettoyage d’écran automatique, effets de texte "machine à écrire" et affichage d’un HUD (Heads-Up Display) centré.
- **Système de Sauvegarde Binaire** : Plutôt que d’utiliser des fichiers textes facilement modifiables par l’utilisateur, nous avons implémenté un système de sauvegarde binaire (`fread/fwrite`). Cela permet de dumper l’état exact de la mémoire (structures entières) dans un fichier `.bin`, rendant la sauvegarde rapide et sécurisée.

1.4 Organisation de l’équipe

Pour mener à bien ce développement, notre équipe s’est organisée de manière agile, avec une répartition des responsabilités claire mais flexible :

- **Julien** s’est chargé de l’architecture globale (le `main` et la boucle de jeu principale), ainsi que de la gestion des tours et des conditions de victoire.
- **Alix** a développé le cœur logique du jeu : la structure des grilles, la gestion des bateaux et surtout l’algorithme de placement valide (vérification des collisions et des débordements).
- **Enzo** a pris en charge l’interface utilisateur (menus, design), le système de sauvegarde et le développement des stratégies de l’Ordinateur.

Des sessions régulières de "Pair Programming" (programmation en binôme) ont été organisées, notamment pour relier l’interface graphique à la logique métier et pour déboguer les pointeurs complexes.

2 Architecture et Planification

Cette section détaille l'organisation structurelle du projet ainsi que la méthodologie de travail adoptée pour respecter les délais imposés.

2.1 Organisation Modulaire (Arborescence)

Pour répondre aux exigences de maintenabilité et éviter un fichier source unique trop volumineux, nous avons opté pour une architecture strictement modulaire. Chaque fichier possède une responsabilité unique, ce qui limite les effets de bord lors des modifications.

Le projet s'articule autour des fichiers suivants :

- **main.c** : Point d'entrée du programme. Il se charge uniquement d'initialiser le générateur de nombres aléatoires (**srand**) et de lancer le Menu Principal.
- **librairies.h** : Fichier d'en-tête "Hub" inclus par tous les autres modules. Il regroupe les inclusions standards (**stdio.h**, **stdlib.h**), les macros de compatibilité système (Windows/Linux) et les codes couleurs ANSI.
- **affichage.c** / **.h** : Gère exclusivement les entrées/sorties avec l'utilisateur (Menus, Dessin des cadres, Effets visuels).
- **jeu.c** / **.h** : Le chef d'orchestre. Il contient la boucle principale du jeu, la gestion de l'alternance des tours et la logique de victoire.
- **grille.c** / **.h** : Responsable de la gestion mémoire des tableaux à deux dimensions (allocation, initialisation, libération).
- **bateau.c** / **.h** : Définit les structures des navires et contient l'algorithme critique de placement (vérification des collisions).
- **sauvegarde.c** / **.h** : Gère la persistance des données via l'écriture et la lecture de fichiers binaires.

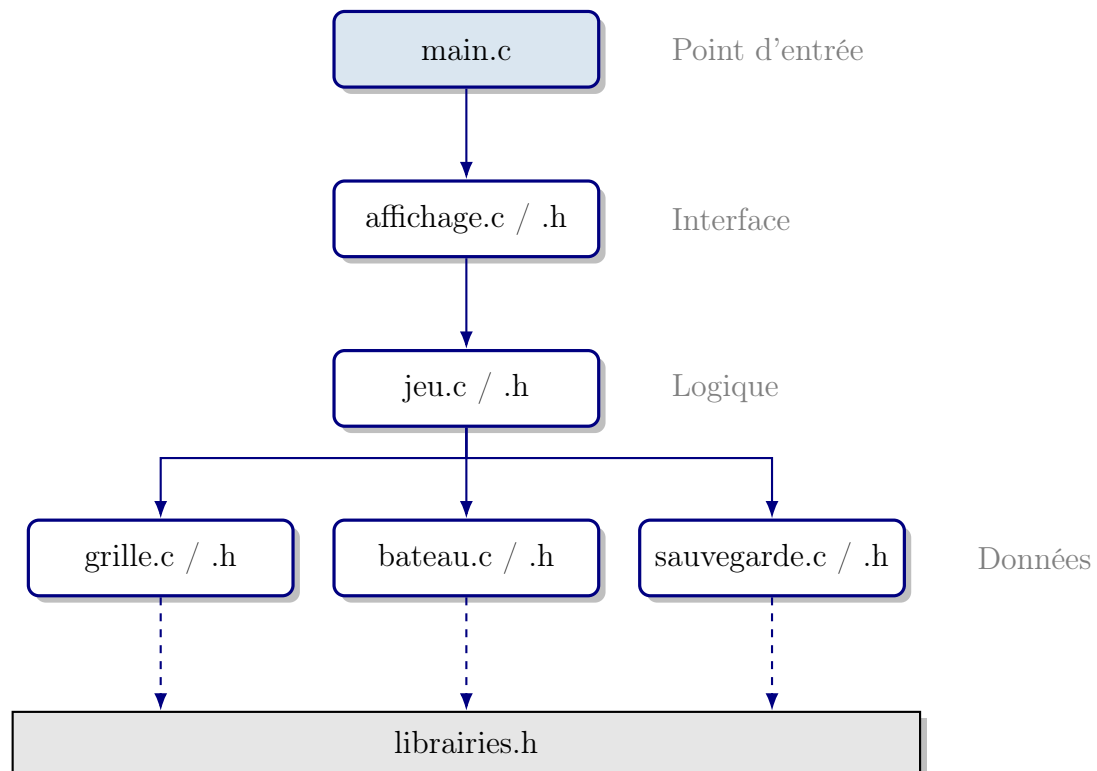


FIGURE 1 – Architecture modulaire et dépendances du projet

2.2 Planification et Répartition des tâches

Le développement s'est étalé sur une période de 14 semaines, du 6 octobre 2025 au 11 janvier 2026. Nous avons adopté une méthode de travail agile, avec des points d'étape hebdomadaires pour fusionner nos travaux respectifs.

Le tableau ci-dessous résume la chronologie du développement :

Période	Tâche Principale	État
06/10 – 19/10	Analyse du sujet, définition des structures de données (Joueur, Grille).	Fait
20/10 – 02/11	Développement des modules "noyau" : Allocation dynamique des grilles, affichage brut et définition de la flotte.	Fait
03/11 – 16/11	Algorithmique de placement : Gestion des collisions et des débordements de grille. Logique de tir de base. Mise en place du dépôt Framagit.	Fait
17/11 – 30/11	Implémentation de la boucle de jeu principale (Mode 1vs1), alternance des tours et détection de fin de partie.	Fait
01/12 – 14/12	Développement de l'Intelligence Artificielle (Mode Aléatoire et Mode Chasseur).	Fait
15/12 – 28/12	Implémentation du système de sauvegarde/-chargement binaire et menus de gestion des slots.	Fait
29/12 – 04/01	Amélioration de l'interface (HUD, Couleurs ANSI) et nettoyage du code (Compilation -Wall -Wextra).	Fait
05/01 – 11/01	Tests de non-régression, vérification des fuites mémoire et rédaction du rapport.	Fait

TABLE 1 – Calendrier prévisionnel et réel du projet

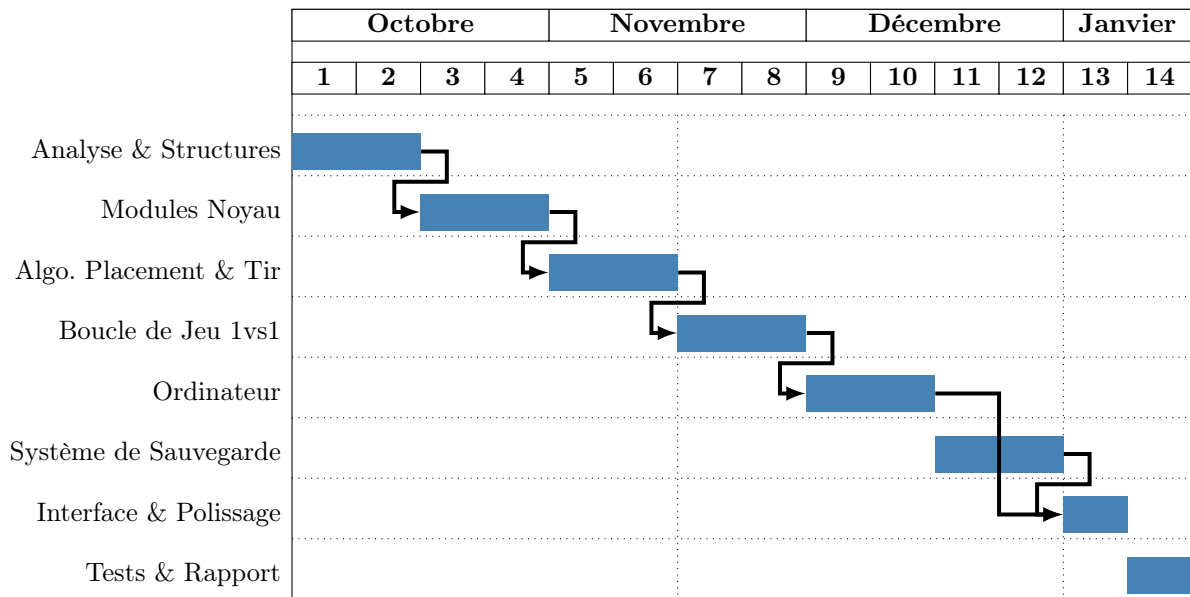


FIGURE 2 – Diagramme de Gantt de la réalisation du projet (Octobre - Janvier)

3 Structures de Données et Logique Cœur

3.1 La structure Joueur : La clé de voûte

La complexité principale de la Bataille Navale réside dans la gestion des états : il faut savoir où sont les bateaux, mais aussi où le joueur a déjà tiré. Pour simplifier les signatures de nos fonctions, nous avons regroupé toutes ces informations dans une structure unique `Joueur` définie dans `jeu.h`.

Cette structure contient une subtilité majeure : l'utilisation de **deux grilles distinctes** pour gérer le "Brouillard de Guerre".

- `Grille* mesNavires` : C'est la grille "vérité". Elle contient la position exacte de la flotte du joueur. C'est sur cette grille que l'adversaire effectue ses tirs.
- `Grille* mesTirs` : C'est la grille "mémoire". Elle est initialement vide et se remplit au fur et à mesure des tirs du joueur (Touché / Raté). C'est celle qui est affichée au joueur lors de son tour offensif.

3.2 Algorithme de Placement des Bateaux

L'une des fonctions les plus critiques du projet est `positionnerBateau` dans `bateau.c`. Elle doit garantir l'intégrité du jeu en empêchant deux situations illégales :

1. Le débordement : Un bateau placé en J10 verticalement sortirait de la grille.
2. La collision : Un bateau ne peut pas être placé sur une case déjà occupée.

Voici le pseudo-code de la logique de validation que nous avons implémentée :

Pseudo-code : Algorithme de positionnement interactif

```
1 FONCTION PositionnerBateau(Bateau *b, Grille *g)
```

```
2
```



```
3 REPETER INDEFINIMENT
4 // 1. Saisie de l'orientation
5 Afficher "Orientation (H/V) ou -1 pour quitter ?"
6 Lire orientation
7 SI (orientation == "-1") RETOURNER 0 // Code d'annulation
8
9 // 2. Saisie des coordonnees
10 Afficher "Coordonnees (ex: A5) ?"
11 Lire buffer
12 colonne = ConvertirLettreEnIndex(buffer[0]) // 'A' -> 0, 'B'
    '-> 1...
13 ligne = ConvertirChiffreEnIndex(buffer[1]) // '1' -> 0,
    '2' -> 1...
14
15 // 3. Verification des limites (Hors Grille)
16 SI (colonne < 0 OU colonne > 9 OU ligne < 0 OU ligne > 9)
17     Afficher "Erreur : Hors grille"
18     CONTINUER (Recommencer la boucle)
19
20 // 4. Verification du debordement (Taille du bateau)
21 SI (Orientation == H ET colonne + taille > 10) OU
22     (Orientation == V ET ligne + taille > 10) ALORS
23     Afficher "Erreur : Le bateau depasse"
24     CONTINUER
25
26 // 5. Verification des collisions (Case deja occupee ?)
27 Collision = FAUX
28 POUR k de 0 a taille-1 FAIRE
29     CaseCible = (Orientation == H) ? [ligne][col + k] : [
        ligne + k][col]
30     SI (g->tab[CaseCible] != 0) ALORS
31         Collision = VRAI
32
33 SI (Collision == VRAI) ALORS
34     Afficher "Erreur : Collision detectee"
35     CONTINUER
36
37 // --- SUCCES : Mises a jour ---
38
39 // A. On marque les cases sur la grille visuelle
40 POUR k de 0 a taille-1 FAIRE
41     CaseCible = (Orientation == H) ? [ligne][col + k] : [
        ligne + k][col]
42     g->tab[CaseCible] = 1 // 1 = Bateau present
43
44 // B. On sauvegarde les coordonnees dans la structure du
    bateau
45 b->ligne = ligne
46 b->colonne = colonne
```

```

47         b->orientation = orientation
48
49         RETOURNER 1 // Succes
50     FIN REPETER
51 FIN FONCTION

```

Cette double vérification (Simulation avant Écriture) est essentielle pour ne pas corrompre la grille avec un bateau à moitié placé qui rencontrerait un obstacle.

4 Fonctionnalités Avancées

4.1 L'Intelligence de l'Ordinateur

Pour rendre le mode "Contre Ordinateur" intéressant, nous ne pouvions pas nous contenter d'un tir purement aléatoire. Nous avons implémenté trois niveaux de difficulté dans `jeu.c` :

- **Facile** : Tir aléatoire pur (en vérifiant simplement de ne pas tirer deux fois au même endroit).
- **Moyen (Chasseur)** : C'est l'algorithme le plus intéressant. L'ordinateur mémorise ses tirs. Dès qu'il touche un navire sans le couler, il passe en mode "Chasse" et cible prioritairement les cases adjacentes (Haut, Bas, Gauche, Droite) pour trouver le reste du bateau.
- **Difficile (Tricheur)** : À des fins de test et de défi, ce niveau de difficulté lit directement la grille de l'adversaire pour trouver les navires.

Voici la logique simplifiée de l'IA "Moyen" :

Pseudo-code : IA Chasseur (Niveau Moyen)

```

1  FONCTION OrdiMoyen(Joueur *ordi, Joueur *humain)
2
3      // 1. Initialisation
4      AdjL = {-1, 1, 0, 0} // Vecteurs Ligne
5      AdjC = { 0, 0, -1, 1} // Vecteurs Colonne
6      CibleTrouvee = FAUX
7      Ligne = -1, Colonne = -1
8
9      // 2. SCAN INTELLIGENT : Chercher une case "Touchee" (3) avec
10     un voisin "Inconnu" (0)
11     POUR i de 0 a 9 FAIRE
12         POUR j de 0 a 9 FAIRE
13             SI (ordi->mesTirs[i][j] == TOUCHE) ALORS
14
15                 // On verifie les 4 voisins
16                 POUR k de 0 a 3 FAIRE
17                     TestL = i + AdjL[k]
18                     TestC = j + AdjC[k]
19
20                     // Si voisin valide (dans la grille) ET pas
21                     encore visite

```

```

20         SI (EstDansGrille(TestL, TestC) ET ordi->
21             mesTirs[TestL][TestC] == 0)
22             Ligne = TestL
23             Colonne = TestC
24             CibleTrouvee = VRAI
25             ALLER A FIN_RECHERCHE // Sortie rapide des
26                                     boucles (goto)
27         FIN SI
28     FIN POUR
29     FIN SI
30     FIN POUR
31     FIN_RECHERCHE:
32
33     // 3. RANDOM : Si aucune cible intelligente trouvee
34     SI (CibleTrouvee == FAUX) ALORS
35         REPETER
36             Ligne = Random(0..9)
37             Colonne = Random(0..9)
38             TANT QUE (ordi->mesTirs[Ligne][Colonne] != 0) // Tant qu'on
39                 a deja joue la
40         FIN SI
41
42     // 4. ACTION DE TIR
43     Resultat = Tirer(humain->mesNavires, Ligne, Colonne)
44
45     SI (Resultat == TOUCHE) ALORS
46         ordi->mesTirs[Ligne][Colonne] = 3 // Marquer "Touche"
47
48         // Mise a jour structurelle du bateau adverse (Vital pour
49             la victoire)
50         Index = TrouverBateau(humain->flotte, Ligne, Colonne)
51         SI (Index != -1)
52             EnregistrerTouche(humain->flotte[Index])
53             SI (EstCoule(humain->flotte[Index]))
54                 Afficher "Bateau Coule !"
55         SINON
56             ordi->mesTirs[Ligne][Colonne] = 2 // Marquer "Rate"
57         FIN SI
58     FIN FONCTION

```

4.2 La Persistance des Données (Sauvegarde)

Pour permettre aux joueurs d'interrompre et de reprendre une partie, nous avons implémenté un système de sauvegarde. Nous avons fait le choix technique d'utiliser des fichiers binaires (.bin) plutôt que des fichiers textes (.txt).

Qu'est-ce que la sauvegarde binaire ?

Contrairement à un fichier texte qui stocke une représentation lisible par un humain (où le nombre 123 est converti en trois caractères '1', '2', '3'), un fichier binaire stocke les données sous leur forme brute, telles qu'elles existent dans la mémoire vive (RAM) de l'ordinateur. Concrètement, cela revient à prendre une "photo" exacte des octets composant nos structures `Joueur` et à les copier directement sur le disque dur.

Dans `sauvegarde.c`, nous utilisons la fonction `fwrite`. Elle nous permet de copier l'intégralité du bloc mémoire de la structure directement dans le fichier, sans aucune conversion :

Exemple : Écriture brute de la mémoire

```
1 // Sauvegarde binaire : on copie la structure entiere d'un coup
2 // fwrite(pointeur_donnee, taille_unitaire, nombre, fichier)
3
4 // Sauvegarde de la grille des navires (tableau 10x10 d'entiers)
5 fwrite(j1->mesNavires, sizeof(Grille), 1, f);
6
7 // Sauvegarde de la flotte (tableau de 5 structures Bateaux)
8 fwrite(j1->flotte, sizeof(Bateaux), NB_BATEAUX, f);
```

Cette méthode présente trois avantages majeurs pour notre projet :

1. **Intégrité** : Nous sommes certains de recharger exactement les mêmes données (pointeurs, états, compteurs) que celles sauvegardées.
2. **Performance** : L'ordinateur ne perd pas de temps à convertir les données en texte puis à les analyser (parsing) au chargement.
3. **Sécurité** : Le fichier produit est illisible pour un utilisateur lambda, ce qui empêche la triche (modifier ses PV ou déplacer un bateau en ouvrant le fichier dans le Bloc-notes).

5 Interface et Ergonomie

5.1 Gestion de l'affichage console

L'un des défis du C en console est de rendre l'affichage lisible. Nous avons utilisé les séquences d'échappement ANSI définies dans `librairies.h` pour colorer la sortie. Cela permet au joueur de distinguer instantanément :

- L'Eau (Bleu)
- Les Touches (Rouge)
- Les Navires (Vert)

De plus, nous avons implémenté un **HUD (Heads-Up Display)** dynamique qui reste affiché en haut de l'écran à chaque tour, indiquant le numéro du tour, le chronomètre et le nom du joueur actif.



FIGURE 3 – Vue Défensive : La grille *Mes Navires* montrant le positionnement de la flotte.

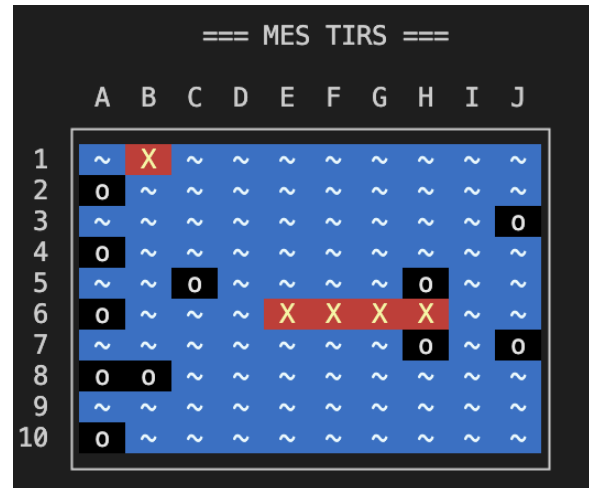


FIGURE 4 – Vue Offensive : La grille *Mes Tirs* avec le marquage des coups (X pour touché, O pour raté).

5.2 Sécurisation des Entrées

Un problème récurrent en C est le plantage du programme lorsqu'un utilisateur entre une lettre alors qu'un chiffre est attendu (via `scanf`).

Pour pallier cela, nous avons mis en place une boucle de nettoyage du "buffer" d'entrée dans tous nos menus :

```

1  if (scanf("%d", &choix) != 1) {
2      int c;
3      // Vide le buffer jusqu'au saut de ligne
4      while ((c = getchar()) != '\n' && c != EOF) {}
5      choix = -1;
6  }

```

Cette sécurité garantit que le jeu ne rentre jamais dans une boucle infinie d'erreur, améliorant grandement l'expérience utilisateur finale.

6 Bilan et Retour d'expérience

6.1 Bilan des réalisations

Au terme de ce projet, nous avons abouti à un jeu de **Bataille Navale** fonctionnelle, robuste et conforme au cahier des charges initial. Le jeu permet des affrontements fluides, que ce soit entre deux joueurs ou contre un ordinateur doté de plusieurs niveaux de difficulté.

Nous sommes particulièrement heureux d'avoir réussi à implémenter des fonctionnalités avancées qui dépassent le cadre du jeu console classique :

- L'architecture modulaire garantit une séparation claire entre l'affichage, la logique métier et les données.
- Le système de sauvegarde binaire est opérationnel et performant.
- L'interface utilisateur, bien qu'en mode texte, offre un confort visuel grâce à l'utilisation intelligente des couleurs et du positionnement du curseur.

6.2 Apports pédagogiques et techniques

Ce projet a été l'occasion de confronter la théorie vue en cours à la réalité du développement. Nous avons pu consolider notre maîtrise de plusieurs concepts clés du langage C :

- **La gestion de la mémoire** : L'utilisation intensive de `malloc` et `free` pour les structures dynamiques (Joueurs, Grilles) nous a sensibilisés aux risques de fuites de mémoire. L'usage d'outils de débogage a été nécessaire pour traquer les erreurs de segmentation.
- **Les fichiers binaires** : Passer de la simple lecture de texte (`fprintf`) à la manipulation de blocs mémoire bruts (`fwrite`) nous a permis de mieux comprendre comment les données sont stockées physiquement sur le disque.
- **L'algorithmique** : La conception des parties contre l'ordinateur, et plus particulièrement l'algorithme de "chasse" (ciblage des cases adjacentes), a demandé une réflexion logique pour traduire une stratégie humaine en code.

6.3 Retour sur le travail d'équipe

La dimension collaborative a été l'un des aspects les plus enrichissants de ce projet. Travailler à trois sur une base de code commune nous a obligés à :

- **Normaliser notre code** : Adopter des conventions de nommage communes pour les variables et les fonctions afin que le code d'Enzo soit lisible par Alix ou Julien.
- **Communiquer** : La répartition des tâches (Interface / Logique / Système) a bien fonctionné, mais a nécessité des points réguliers pour s'assurer que les modules s'interfaçaient correctement (ex : que la fonction d'affichage reçoive les bonnes structures de la part du moteur de jeu).

6.4 Perspectives d'amélioration

Si nous disposions de plus de temps, plusieurs pistes d'évolution pourraient être explorées pour enrichir le projet :

- **Mode Réseau** : Remplacer le mode "1vs1 local" par une communication via Sockets TCP/IP pour permettre à deux joueurs de s'affronter depuis des machines différentes.
- **Interface Graphique** : Porter le jeu vers une bibliothèque graphique comme la SDL2 ou Raylib pour sortir de la console et proposer une expérience visuelle plus moderne (souris, sprites).
- **Son** : Ajouter des effets sonores (bruitages de tirs, ploufs, explosions) pour renforcer l'immersion.

En conclusion, ce projet de Bataille Navale a été une expérience formatrice complète, nous permettant de passer du statut d'étudiants apprenant la syntaxe du C à celui d'étudiants capables de structurer et mener à bien un projet logiciel complexe.